

13-Tarefa: Avaliação de Afinidade

Metodologia

Nesta tarefa, testamos o impacto de diferentes afinidades de *threads* geradas pelo OpenMP no desempenho do algoritmo solver de Navier-Stokes. Utilizamos um processador multicore (testado no macOS M-series / Apple Silicon para referência local com 16 threads via OMP).

Afinidade de *threads* diz respeito à forma como as threads de execução estão amarradas (bind) à hierarquia física do processador (Cores, Sockets, Hardware Threads).

As diretivas avaliadas foram:

- **OMP_PLACES:** cores, sockets, threads
- **OMP_PROC_BIND:** `close` (threads adjacentes entre si) e `spread` (espalhadas para distribuição equilibrada nas caches e controle térmico).

Resultados Obtidos

Abaixo apresentamos os resultados coletados com 16 threads executando a versão `static`:

Threads	Afinidade (Places / Bind)	Tempo (s)	Speedup (T1/Tn)
1	Padrão (Serial)	2.509	1.00x
16	Padrão	0.765	3.28x
16	cores / <code>close</code>	0.656	3.82x
16	cores / <code>spread</code>	0.610	4.11x
16	sockets / <code>close</code>	0.624	4.02x
16	sockets / <code>spread</code>	0.605	4.14x
16	threads / <code>close</code>	0.579	4.33x
16	threads / <code>spread</code>	0.589	4.26x

Discussão

Através dos resultados, podemos destacar os seguintes pontos:

- **close vs spread:** Em quase todos os agrupamentos (places), a estratégia `spread` apresentou resultados ligeiramente melhores ou equivalentes em comparação com `close`. Isso ocorre porque `spread` tende a distribuir melhor

o trabalho pela hierarquia do sistema, evitando contenção exagerada de caches de mesmo nível (L1/L2) e balanceando o aspecto térmico entre os núcleos reais. Contudo, na configuração de threads, o `close` foi mais rápido.

- **cores vs threads:** A afinidade atrelada diretamente às *threads* físicas apresentou o melhor tempo e speedup absoluto (4.33x com `close`). Ao amarrar estritamente o worker na thread computacional ao invés do core, reduzimos ainda mais o custo de *context switch* do sistema operacional. O speedup não escalou muito mais do que ~4.3x devido ao tamanho do problema restrito (1000 iterações/pequena malha) limitando o ganho de 16 threads em detrimento do custo de sincronização das barreiras implícitas OpenMP.
- Em todos os cenários as regras de afinidades mostraram uma evidente otimização em relação a execução Padrão.

Apêndice

```
#!/bin/bash
#SBATCH --job-name=navier_afinidade
#SBATCH --time=0-1:00
#SBATCH --partition=intel-128
#SBATCH --nodes=1
#SBATCH --cpus-per-task=16
#SBATCH --hint=compute_bound

export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
make clean
make

EXEC="./navier_static"
PLACES=("cores" "sockets" "threads")
BINDS=("close" "spread")

for THREADS in 4 8 16; do
    export OMP_NUM_THREADS=$THREADS
    export OMP_PROC_BIND=false
    export OMP_PLACES=
    $EXEC $THREADS

    for PLACE in "${PLACES[@]"; do
        for BIND in "${BINDS[@]"; do
```

```
export OMP_PLACES=$PLACE
export OMP_PROC_BIND=$BIND
$EXEC $THREADS
done
done
done
```